**CSE 579**
**Simple Answer Set Programming Graded Assignment**
**Template for clingo Work**

Problem 1

| | |
|---|---|
| Input Program | % Problem 1: 8-Queens with NO queens in the middle 4x4<br>#const n=8.<br>% Each row has exactly one queen (row constraint)<br>{ queen(R,1..n) } = 1 :- R=1..n.<br>% No two queens are on the same column (column constraint)<br>:- queen(R1,C), queen(R2,C), R1 != R2.<br>% No two queens are on the same diagonal (diagonal constraint)<br>:- queen(R1,C1), queen(R2,C2), R1 != R2, \|R1-R2\| = \|C1-C2\|.<br>% No queens in the middle 4x4 (rows 3..6, cols 3..6) (middle board constraint)<br>:- queen(R,C), R=3..6, C=3..6.<br>#show queen/2. |
| Command Line | clingo problem1.txt 0 |
| Output of clingo | clingo version 5.8.0<br>Reading from problem1.txt<br>Solving...<br>Answer: 1 (Time: 0.002s)<br>queen(5,7) queen(1,4) queen(2,6) queen(4,2) queen(3,8) queen(6,1) queen(7,3) queen(8,5)<br>Answer: 2 (Time: 0.005s)<br>queen(2,3) queen(3,1) queen(6,8) queen(4,7) queen(1,5) queen(5,2) queen(7,6) queen(8,4)<br>Answer: 3 (Time: 0.006s)<br>queen(2,4) queen(4,1) queen(5,8) queen(3,7) queen(1,6) queen(6,2) queen(7,5) queen(8,3)<br>Answer: 4 (Time: 0.007s)<br>queen(6,7) queen(1,3) queen(2,5) queen(3,2) queen(4,8) queen(5,1) queen(8,6) queen(7,4)<br>SATISFIABLE<br><br>Models : 4<br>Calls : 1<br>Time : 0.008s (Solving: 0.01s 1st Model: 0.00s Unsat: 0.00s)<br>CPU Time : 0.000s |

Problem 2

**Input Program**

```
% Problem 2: n-Queens (parameterized by n)
#const n=8.
% Each row has exactly one queen (row constraint)
{ queen(R,1..n) } = 1 :- R=1..n.
% No two queens are on the same column (column constraint)
:- queen(R1,C), queen(R2,C), R1 != R2.
% No two queens are on the same diagonal (diagonal constraint)
:- queen(R1,C1), queen(R2,C2), R1 != R2, |R1-R2| = |C1-C2|.
% (Do NOT include the middle-board constraint for Problem 2)
% Optional: hide solutions to keep output small
#show.
```

**Command Line**

You should write 10 command lines below, one for each value of n from the set {3, …, 12}.

```
clingo problem2.txt -c n=3 0
clingo problem2.txt -c n=4 0
clingo problem2.txt -c n=5 0
clingo problem2.txt -c n=6 0
clingo problem2.txt -c n=7 0
clingo problem2.txt -c n=8 0
clingo problem2.txt -c n=9 0
clingo problem2.txt -c n=10 0
clingo problem2.txt -c n=11 0
clingo problem2.txt -c n=12 0
```

**Output of clingo**

Since there are 10 outputs for 10 different values of n, please do not copy and paste the outputs of clingo for Problem 2 (and also for Problem 8 later).

**Answer to Questions**

Draw a table that lists the number of solutions and the times to compute all solutions. Use CPU time that clingo returns.

| Value n | Number of solutions | time |
|---|---|---|
| 3 | 0 | 0.000 |
| 4 | 2 | 0.000 |
| 5 | 10 | 0.000 |
| 6 | 4 | 0.000 |
| 7 | 40 | 0.000 |
| 8 | 92 | 0.000 |
| 9 | 352 | 0.031 |
| 10 | 724 | 0.156 |
| 11 | 2680 | 1.563 |
| 12 | 14200 | 27.906 |

Problem 3

| | |
|---|---|
| Input Program | % Problem 3: Sudoku (World's hardest Sudoku - Arto Inkala puzzle)<br>#const n=9.<br>% Define potential solutions:<br>% a(R,C,D) means digit D is placed in row R, column C.<br>% Each digit 1..9 appears exactly once in each 3x3 subgrid (box constraint)<br>1 { a(R,C,D) : R=1..9, C=1..9,<br>R0 <= R, R <= R0+2,<br>C0 <= C, C <= C0+2 } 1<br>:- D=1..9, R0 = 3*(0..2)+1, C0 = 3*(0..2)+1.<br>% Cell constraint (one value per cell)<br>:- a(R,C,D1), a(R,C,D2), D1 != D2.<br>% Row constraint (same digit cannot appear twice in a row)<br>:- a(R,C1,D), a(R,C2,D), C1 != C2.<br>% Column constraint (same digit cannot appear twice in a column)<br>:- a(R1,C,D), a(R2,C,D), R1 != R2.<br>% Givens (the puzzle clues)<br>% Write them as g(R,C,D). Then force them into the solution.<br>:- g(R,C,D), not a(R,C,D).<br>% Instance for Problem 3<br>g(1,1,8).<br>g(2,3,3). g(2,4,6).<br>g(3,2,7). g(3,5,9). g(3,7,2).<br>g(4,2,5). g(4,6,7).<br>g(5,5,4). g(5,6,5). g(5,7,7).<br>g(6,4,1). g(6,8,3).<br>g(7,3,1). g(7,8,6). g(7,9,8).<br>g(8,3,8). g(8,4,5). g(8,8,1).<br>g(9,2,9). g(9,7,4).<br>% Output<br>#show a/3. |
| Command Line | clingo problem3.txt 0 |
| Output of clingo | clingo version 5.8.0<br>Reading from problem3.txt<br>Solving...<br>Answer: 1 (Time: 0.026s)<br>a(1,1,8) a(2,3,3) a(2,4,6) a(3,2,7) a(3,5,9) a(3,7,2) a(4,2,5) a(4,6,7) a(5,5,4) a(5,6,5) a(5,7,7) a(6,4,1) a(6,8,3) a(7,3,1) a(7,8,6) a(7,9,8) a(8,3,8) a(8,4,5) a(8,8,1) a(9,2,9) a(9,7,4) a(4,1,1) a(1,2,1) a(6,1,2) a(7,2,2) a(1,3,2) a(5,1,3) a(8,2,3) a(8,1,4) a(2,2,4) a(4,3,4) a(7,1,5) a(3,3,5) a(3,1,6) a(5,2,6) a(9,3,6) a(9,1,7) a(6,3,7) a(6,2,8) a(2,1,9) a(5,3,9) a(9,5,1) a(3,6,1) a(4,4,2) a(8,5,2) a(2,6,2) a(9,4,3) a(4,5,3) a(1,6,3) a(3,4,4) a(7,6,4) a(1,5,5) a(6,5,6) a(8,6,6) a(1,4,7) a(7,5,7) a(5,4,8) a(2,5,8) a(9,6,8) a(7,4,9) a(6,6,9) a(2,7,1) a(5,9,1) a(5,8,2) a(9,9,2) a(7,7,3) a(3,9,3) a(1,8,4) a(6,9,4) a(6,7,5) a(9,8,5) a(2,9,5) a(1,7,6) a(4,9,6) a(2,8,7) a(8,9,7) a(4,7,8) a(3,8,8) a(8,7,9) a(4,8,9) a(1,9,9)<br>SATISFIABLE<br><br>Models : 1<br>Calls : 1<br>Time : 0.036s (Solving: 0.01s 1st Model: 0.00s Unsat: 0.01s)<br>CPU Time : 0.016s |

Problem 4

| | |
|---|---|
| Input Program | % the facts to represent the given numbers are provided below<br>a(1,1,9). a(1,2,14). a(1,6,3). a(1,8,5). a(1,9,15). a(1,11,2). a(1,15,7). a(1,16,1).<br>a(2,1,6). a(2,2,12). a(2,6,14). a(2,11,10). a(2,15,5). a(2,16,11).<br>a(3,1,4). a(3,4,7). a(3,5,6). a(3,8,13). a(3,9,16). a(3,12,1). a(3,13,2). a(3,16,9).<br>a(4,2,15). a(4,3,16). a(4,5,9). a(4,6,7). a(4,11,11). a(4,12,6). a(4,14,3). a(4,15,14).<br>a(5,2,7). a(5,3,15). a(5,14,2). a(5,15,16).<br>a(6,1,5). a(6,3,13). a(6,5,14). a(6,7,15). a(6,10,10). a(6,12,3). a(6,14,1). a(6,16,8).<br>a(7,2,8). a(7,4,10). a(7,6,9). a(7,7,4). a(7,8,11). a(7,9,13). a(7,10,6). a(7,11,15). a(7,13,14).<br>a(7,15,3).<br>a(8,1,16). a(8,5,5). a(8,7,3). a(8,10,14). a(8,12,9). a(8,16,6).<br>a(9,1,15). a(9,5,16). a(9,7,10). a(9,10,9). a(9,12,13). a(9,16,14).<br>a(10,2,9). a(10,4,6). a(10,6,5). a(10,7,13). a(10,8,3). a(10,9,1). a(10,10,15). a(10,11,4).<br>a(10,13,7). a(10,15,12).<br>a(11,1,2). a(11,3,8). a(11,5,15). a(11,7,14). a(11,10,16). a(11,12,12). a(11,14,5).<br>a(11,16,13).<br>a(12,2,13). a(12,3,12). a(12,14,9). a(12,15,11).<br>a(13,2,5). a(13,3,3). a(13,5,2). a(13,6,16). a(13,11,13). a(13,12,10). a(13,14,12). a(13,15,9).<br>a(14,1,8). a(14,4,4). a(14,5,12). a(14,8,1). a(14,9,6). a(14,12,7). a(14,13,15). a(14,16,3).<br>a(15,1,10). a(15,2,1). a(15,6,15). a(15,11,16). a(15,15,6). a(15,16,2).<br>a(16,1,11). a(16,2,2). a(16,6,8). a(16,8,14). a(16,9,3). a(16,11,1). a(16,15,10). a(16,16,7).<br>% write the remaining part of the clingo program below<br>% Problem 4: 16x16 Sudoku<br>#const n=16.<br>% Define potential solutions:<br>% x(R,C,V) means value V is placed in row R, column C.<br>% Each value appears exactly once in each 4x4 subgrid<br>1 { x(R,C,V) : R=1..n, C=1..n,<br>R0 <= R, R <= R0+3,<br>C0 <= C, C <= C0+3 } 1<br>:- V=1..n, R0 = 4*(0..3)+1, C0 = 4*(0..3)+1.<br>% Cell constraint (one value per cell)<br>:- x(R,C,V1), x(R,C,V2), V1 != V2.<br>% Row constraint<br>:- x(R,C1,V), x(R,C2,V), C1 != C2.<br>% Column constraint<br>:- x(R1,C,V), x(R2,C,V), R1 != R2.<br>% Force the given clues<br>:- a(R,C,V), not x(R,C,V).<br>#show x/3. |
| Command Line | clingo problem4.txt 0 |
| Output of clingo | clingo version 5.8.0<br>Reading from problem4.txt<br>Solving...<br>Answer: 1 (Time: 0.176s)<br>x(1,1,9) x(1,2,14) x(1,6,3) x(1,8,5) x(1,9,15) x(1,11,2) x(1,15,7) x(1,16,1) x(2,1,6) x(2,2,12)<br>x(2,6,14) x(2,11,10) x(2,15,5) x(2,16,11) x(3,1,4) x(3,4,7) x(3,5,6) x(3,8,13) x(3,9,16)<br>x(3,12,1) x(3,13,2) x(3,16,9) x(4,2,15) x(4,3,16) x(4,5,9) x(4,6,7) x(4,11,11) x(4,12,6)<br>x(4,14,3) x(4,15,14) x(5,2,7) x(5,3,15) x(5,14,2) x(5,15,16) x(6,1,5) x(6,3,13) x(6,5,14)<br>x(6,7,15) x(6,10,10) x(6,12,3) x(6,14,1) x(6,16,8) x(7,2,8) x(7,4,10) x(7,6,9) x(7,7,4) x(7,8,11) |

| | |
|---|---|
| | x(7,9,13) x(7,10,6) x(7,11,15) x(7,13,14) x(7,15,3) x(8,1,16) x(8,5,5) x(8,7,3) x(8,10,14) x(8,12,9) x(8,16,6) x(9,1,15) x(9,5,16) x(9,7,10) x(9,10,9) x(9,12,13) x(9,16,14) x(10,2,9) x(10,4,6) x(10,6,5) x(10,7,13) x(10,8,3) x(10,9,1) x(10,10,15) x(10,11,4) x(10,13,7) x(10,15,12) x(11,1,2) x(11,3,8) x(11,5,15) x(11,7,14) x(11,10,16) x(11,12,12) x(11,14,5) x(11,16,13) x(12,2,13) x(12,3,12) x(12,14,9) x(12,15,11) x(13,2,5) x(13,3,3) x(13,5,2) x(13,6,16) x(13,11,13) x(13,12,10) x(13,14,12) x(13,15,9) x(14,1,8) x(14,4,4) x(14,5,12) x(14,8,1) x(14,9,6) x(14,12,7) x(14,13,15) x(14,16,3) x(15,1,10) x(15,2,1) x(15,6,15) x(15,11,16) x(15,15,6) x(15,16,2) x(16,1,11) x(16,2,2) x(16,6,8) x(16,8,14) x(16,9,3) x(16,11,1) x(16,15,10) x(16,16,7) x(12,1,1) x(2,3,1) x(8,4,1) x(7,3,2) x(4,4,2) x(5,1,3) x(9,2,3) x(2,4,3) x(11,2,4) x(8,3,4) x(3,3,5) x(9,4,5) x(6,2,6) x(16,3,6) x(13,1,7) x(9,3,7) x(1,4,8) x(15,3,9) x(6,4,9) x(3,2,10) x(10,3,10) x(8,2,11) x(1,3,11) x(11,4,11) x(7,1,12) x(16,4,12) x(4,1,13) x(15,4,13) x(10,1,14) x(14,3,14) x(5,4,14) x(13,4,15) x(14,2,16) x(12,4,16) x(7,5,1) x(9,6,1) x(4,7,1) x(8,6,2) x(2,7,2) x(12,8,2) x(15,5,3) x(2,5,4) x(12,6,4) x(15,8,4) x(14,7,5) x(11,6,6) x(5,7,6) x(13,8,6) x(12,5,7) x(15,7,7) x(8,8,7) x(5,5,8) x(12,7,8) x(4,8,8) x(16,7,9) x(11,8,9) x(1,5,10) x(14,6,10) x(5,8,10) x(10,5,11) x(3,6,11) x(13,7,11) x(6,6,12) x(3,7,12) x(9,8,12) x(16,5,13) x(5,6,13) x(2,8,15) x(1,7,16) x(6,8,16) x(5,10,1) x(6,9,2) x(14,10,2) x(10,12,2) x(3,10,3) x(11,11,3) x(5,9,4) x(16,10,4) x(1,12,4) x(4,9,5) x(12,10,5) x(5,11,5) x(15,12,5) x(12,11,6) x(11,9,7) x(2,10,7) x(6,11,7) x(8,9,8) x(13,10,8) x(9,11,8) x(2,12,8) x(2,9,9) x(14,11,9) x(12,9,10) x(9,9,11) x(15,10,11) x(5,12,11) x(15,9,12) x(4,10,12) x(8,11,12) x(1,10,13) x(13,9,14) x(3,11,14) x(12,12,14) x(16,12,15) x(7,12,16) x(13,13,1) x(11,15,1) x(9,15,2) x(12,13,3) x(4,13,4) x(9,14,4) x(6,15,4) x(13,16,4) x(16,13,5) x(7,16,5) x(9,13,6) x(1,14,6) x(7,14,7) x(15,13,8) x(10,14,8) x(3,15,8) x(5,13,9) x(11,13,10) x(8,14,10) x(4,16,10) x(6,13,11) x(14,14,11) x(1,13,12) x(5,16,12) x(8,13,13) x(2,14,13) x(14,15,13) x(15,14,14) x(3,14,15) x(8,15,15) x(12,16,15) x(2,13,16) x(16,14,16) x(10,16,16)<br>SATISFIABLE<br><br>Models : 1<br>Calls : 1<br>Time : 0.225s (Solving: 0.05s 1st Model: 0.00s Unsat: 0.05s)<br>CPU Time : 0.172s |

Problem 5

| | |
|---|---|
| Input Program | % the facts to represent the given numbers are provided below<br>a(1,3,7). a(1,7,8).<br>a(2,2,2). a(2,8,4).<br>a(3,1,8). a(3,3,4). a(3,5,2). a(3,7,5). a(3,9,1).<br>a(4,5,7).<br>a(5,3,8). a(5,4,3). a(5,5,6). a(5,6,4). a(5,7,2).<br>a(6,5,9).<br>a(7,1,3). a(7,3,2). a(7,5,8). a(7,7,7). a(7,9,4).<br>a(8,2,7). a(8,8,8).<br>a(9,3,6). a(9,7,9).<br>% write the remaining part of the clingo program below<br>% Define potential solutions for standard Sudoku<br>% Each digit appears exactly once in each 3x3 box<br>1 { x(R,C,N) : R=1..9, C=1..9,<br>R0 <= R, R <= R0+2,<br>C0 <= C, C <= C0+2 } 1<br>:- N=1..9, R0 = 3*(0..2)+1, C0 = 3*(0..2)+1.<br>% Cell constraint (one value per cell)<br>:- x(R,C,N1), x(R,C,N2), N1 != N2.<br>% Row constraint<br>:- x(R,C1,N), x(R,C2,N), C1 != C2.<br>% Column constraint<br>:- x(R1,C,N), x(R2,C,N), R1 != R2.<br>% OFFSET (color) constraint:<br>% Same color = same position inside a 3x3 box (same (R-1) mod 3 and (C-1) mod 3)<br>:- x(R,C,N), x(R1,C1,N),<br>(R-1) \ 3 == (R1-1) \ 3,<br>(C-1) \ 3 == (C1-1) \ 3,<br>(R,C) != (R1,C1).<br>% Force the given clues<br>:- a(R,C,N), not x(R,C,N).<br>#show x/3. |
| Command Line | clingo problem5.txt 0 |
| Output of clingo | clingo version 5.8.0<br>Reading from problem5.txt<br>Solving...<br>Answer: 1 (Time: 0.029s)<br>x(1,3,7) x(1,7,8) x(2,2,2) x(2,8,4) x(3,1,8) x(3,3,4) x(3,5,2) x(3,7,5) x(3,9,1) x(4,5,7) x(5,3,8) x(5,4,3) x(5,5,6) x(5,6,4) x(5,7,2) x(6,5,9) x(7,1,3) x(7,3,2) x(7,5,8) x(7,7,7) x(7,9,4) x(8,2,7) x(8,8,8) x(9,3,6) x(9,7,9) x(1,1,1) x(7,2,1) x(4,3,1) x(8,7,1) x(5,8,1) x(2,6,1) x(6,4,1) x(9,5,1) x(4,1,2) x(1,8,2) x(8,9,2) x(9,4,2) x(6,6,2) x(4,2,3) x(1,6,3) x(8,5,3) x(2,3,3) x(6,7,3) x(3,8,3) x(9,9,3) x(4,7,4) x(1,5,4) x(8,4,4) x(9,1,4) x(6,2,4) x(4,4,5) x(1,2,5) x(7,6,5) x(8,1,5) x(2,5,5) x(5,9,5) x(9,8,5) x(6,3,5) x(1,4,6) x(7,8,6) x(4,9,6) x(2,7,6) x(8,6,6) x(6,1,6) x(3,2,6) x(5,1,7) x(2,9,7) x(3,4,7) x(6,8,7) x(9,6,7) x(4,6,8) x(2,4,8) x(9,2,8) x(6,9,8) x(7,4,9) x(4,8,9) x(1,9,9) x(2,1,9) x(5,2,9) x(8,3,9) x(3,6,9)<br>SATISFIABLE<br><br>Models : 1<br>Calls : 1 |

| | |
|---|---|
| | Time : 0.037s (Solving: 0.01s 1st Model: 0.00s Unsat: 0.01s)<br>CPU Time : 0.047s |

Problem 6

| | |
|---|---|
| Input Program | % the facts to represent the given numbers are provided below<br>a(1,1,3). a(1,9,4).<br>a(2,4,6). a(2,6,9).<br>a(3,3,6). a(3,7,9).<br>a(4,2,8). a(4,4,3). a(4,6,2). a(4,8,6).<br>a(5,5,7).<br>a(6,2,1). a(6,4,8). a(6,6,5). a(6,8,7).<br>a(7,3,7). a(7,7,8).<br>a(8,4,7). a(8,6,8).<br>a(9,1,9). a(9,9,7).<br>% write the remaining part of the clingo program below<br>% Define potential solutions for standard Sudoku<br>% Each digit appears exactly once in each 3x3 box<br>1 { x(R,C,N) : R=1..9, C=1..9,<br>R0 <= R, R <= R0+2,<br>C0 <= C, C <= C0+2 } 1<br>:- N=1..9, R0 = 3*(0..2)+1, C0 = 3*(0..2)+1.<br>% Cell constraint (one value per cell)<br>:- x(R,C,N1), x(R,C,N2), N1 != N2.<br>% Row constraint<br>:- x(R,C1,N), x(R,C2,N), C1 != C2.<br>% Column constraint<br>:- x(R1,C,N), x(R2,C,N), R1 != R2.<br>% ANTI-KNIGHT CONSTRAINT<br>% Cells that are a chess knight's move away cannot hold equal values<br>% Simple version using Manhattan distance<br>:- x(R,C,N), x(R1,C1,N), \|R1-R\|+\|C1-C\|==3.<br>% Force the given clues<br>:- a(R,C,N), not x(R,C,N).<br>#show x/3. |
| Command Line | clingo problem6.txt 0 |
| Output of clingo | clingo version 5.8.0<br>Reading from problem6.txt<br>Solving...<br>Answer: 1 (Time: 0.026s)<br>x(1,1,3) x(1,9,4) x(2,4,6) x(2,6,9) x(3,3,6) x(3,7,9) x(4,2,8) x(4,4,3) x(4,6,2) x(4,8,6) x(5,5,7) x(6,2,1) x(6,4,8) x(6,6,5) x(6,8,7) x(7,3,7) x(7,7,8) x(8,4,7) x(8,6,8) x(9,1,9) x(9,9,7) x(1,3,1) x(3,6,1) x(4,5,1) x(1,5,2) x(2,2,2) x(6,1,2) x(3,5,3) x(5,3,3) x(2,1,4) x(3,4,4) x(6,3,4) x(2,3,5) x(1,4,5) x(5,2,5) x(5,1,6) x(3,2,7) x(4,1,7) x(1,6,7) x(3,1,8) x(2,5,8) x(1,2,9) x(4,3,9) x(5,4,9) x(2,9,1) x(3,9,2) x(5,7,2) x(2,8,3) x(5,6,4) x(4,7,4) x(3,8,5) x(1,7,6) x(6,5,6) x(2,7,7) x(1,8,8) x(5,8,1) x(6,7,3) x(4,9,5) x(5,9,8) x(6,9,9) x(7,1,1) x(8,3,2) x(7,4,2) x(9,2,3) x(7,2,4) x(8,1,5) x(7,5,5) x(8,2,6) x(9,3,8) x(9,4,1) x(8,7,1) x(7,6,3) x(9,5,4) x(9,6,6) x(8,5,9) x(7,8,9) x(9,8,2) x(8,9,3) x(8,8,4) x(9,7,5) x(7,9,6)<br>SATISFIABLE<br><br>Models : 1<br>Calls : 1<br>Time : 0.048s (Solving: 0.02s 1st Model: 0.00s Unsat: 0.02s)<br>CPU Time : 0 |

Problem 7

| Input Program | |
|---|---|

```
% In general, it would be time-consuming to enumerate a lot of similar facts in a domain
% thus we always use rules to help us generate those facts.
% In problem 7, it's difficult to enumerate all greater-than relations without a mistake
% thus we provide some rules that you can start from.
% ================================================================
% define the directions of the greater-than symbols on each row
left_right(1, l, l, r, l, l, l).
left_right(2, l, r, l, r, l, l).
left_right(3, r, l, r, r, r, l).
left_right(4, r, l, l, l, r, l).
left_right(5, l, r, r, l, r, r).
left_right(6, l, r, l, l, l, r).
left_right(7, r, l, r, l, l, r).
left_right(8, r, r, l, r, l, l).
left_right(9, l, r, l, r, l, l).
up_down(1, u, u, d, u, u, d, d, d, d).
up_down(2, u, d, d, d, d, d, u, u, d).
up_down(4, d, u, d, u, u, d, u, u, u).
up_down(5, u, d, u, d, d, u, u, u, u).
up_down(7, d, u, d, d, u, d, d, u, u).
up_down(8, d, u, u, u, u, d, d, d, d).
% define the effect of the greater-than symbols on left-right directions
N1 < N2 :- left_right(R, D, _, _, _, _, _), D=l, a(R,1,N1), a(R,2,N2).
N1 < N2 :- left_right(R, _, D, _, _, _, _), D=l, a(R,2,N1), a(R,3,N2).
N1 < N2 :- left_right(R, _, _, D, _, _, _), D=l, a(R,4,N1), a(R,5,N2).
N1 < N2 :- left_right(R, _, _, _, D, _, _), D=l, a(R,5,N1), a(R,6,N2).
N1 < N2 :- left_right(R, _, _, _, _, D, _), D=l, a(R,7,N1), a(R,8,N2).
N1 < N2 :- left_right(R, _, _, _, _, _, D), D=l, a(R,8,N1), a(R,9,N2).
N1 > N2 :- left_right(R, D, _, _, _, _, _), D=r, a(R,1,N1), a(R,2,N2).
N1 > N2 :- left_right(R, _, D, _, _, _, _), D=r, a(R,2,N1), a(R,3,N2).
N1 > N2 :- left_right(R, _, _, D, _, _, _), D=r, a(R,4,N1), a(R,5,N2).
N1 > N2 :- left_right(R, _, _, _, D, _, _), D=r, a(R,5,N1), a(R,6,N2).
N1 > N2 :- left_right(R, _, _, _, _, D, _), D=r, a(R,7,N1), a(R,8,N2).
N1 > N2 :- left_right(R, _, _, _, _, _, D), D=r, a(R,8,N1), a(R,9,N2).
% you need to write down the remaining program below. For example, the first rule
% should "define the effect of the greater-than symbols on up-down directions"
% Define the effect of the greater-than symbols on up-down directions
% When D=u (up arrow), cell above < cell below
N1 < N2 :- up_down(R, D, _, _, _, _, _, _, _, _), D=u, a(R,1,N1), a(R+1,1,N2).
N1 < N2 :- up_down(R, _, D, _, _, _, _, _, _, _), D=u, a(R,2,N1), a(R+1,2,N2).
N1 < N2 :- up_down(R, _, _, D, _, _, _, _, _, _), D=u, a(R,3,N1), a(R+1,3,N2).
N1 < N2 :- up_down(R, _, _, _, D, _, _, _, _, _), D=u, a(R,4,N1), a(R+1,4,N2).
N1 < N2 :- up_down(R, _, _, _, _, D, _, _, _, _), D=u, a(R,5,N1), a(R+1,5,N2).
N1 < N2 :- up_down(R, _, _, _, _, _, D, _, _, _), D=u, a(R,6,N1), a(R+1,6,N2).
N1 < N2 :- up_down(R, _, _, _, _, _, _, D, _, _), D=u, a(R,7,N1), a(R+1,7,N2).
N1 < N2 :- up_down(R, _, _, _, _, _, _, _, D, _), D=u, a(R,8,N1), a(R+1,8,N2).
N1 < N2 :- up_down(R, _, _, _, _, _, _, _, _, D), D=u, a(R,9,N1), a(R+1,9,N2).

% When D=d (down arrow), cell above > cell below
N1 > N2 :- up_down(R, D, _, _, _, _, _, _, _, _), D=d, a(R,1,N1), a(R+1,1,N2).
```

| | |
|---|---|
| | N1 > N2 :- up_down(R, _, D, _, _, _, _, _, _, _), D=d, a(R,2,N1), a(R+1,2,N2).<br>N1 > N2 :- up_down(R, _, _, D, _, _, _, _, _, _), D=d, a(R,3,N1), a(R+1,3,N2).<br>N1 > N2 :- up_down(R, _, _, _, D, _, _, _, _, _), D=d, a(R,4,N1), a(R+1,4,N2).<br>N1 > N2 :- up_down(R, _, _, _, _, D, _, _, _, _), D=d, a(R,5,N1), a(R+1,5,N2).<br>N1 > N2 :- up_down(R, _, _, _, _, _, D, _, _, _), D=d, a(R,6,N1), a(R+1,6,N2).<br>N1 > N2 :- up_down(R, _, _, _, _, _, _, D, _, _), D=d, a(R,7,N1), a(R+1,7,N2).<br>N1 > N2 :- up_down(R, _, _, _, _, _, _, _, D, _), D=d, a(R,8,N1), a(R+1,8,N2).<br>N1 > N2 :- up_down(R, _, _, _, _, _, _, _, _, D), D=d, a(R,9,N1), a(R+1,9,N2).<br>% Define potential solutions for standard Sudoku<br>% Each digit appears exactly once in each 3x3 box<br>1 { a(R,C,N) : R=1..9, C=1..9,<br>R0 <= R, R <= R0+2,<br>C0 <= C, C <= C0+2 } 1<br>:- N=1..9, R0 = 3*(0..2)+1, C0 = 3*(0..2)+1.<br>% Cell constraint (one value per cell)<br>:- a(R,C,N1), a(R,C,N2), N1 != N2.<br>% Row constraint<br>:- a(R,C1,N), a(R,C2,N), C1 != C2.<br>% Column constraint<br>:- a(R1,C,N), a(R2,C,N), R1 != R2.<br>#show a/3. |
| Command Line | clingo problem7.txt 0 |
| Output of clingo | clingo version 5.8.0<br>Reading from problem7.txt<br>Solving...<br>Answer: 1 (Time: 0.119s)<br>a(5,1,1) a(3,2,1) a(8,3,1) a(1,1,2) a(7,2,2) a(5,3,2) a(6,1,3) a(1,2,3) a(9,3,3) a(2,1,4) a(8,2,4) a(6,3,4) a(9,1,5) a(6,2,5) a(3,3,5) a(8,1,6) a(4,2,6) a(2,3,6) a(7,1,7) a(2,2,7) a(4,3,7) a(3,1,8) a(5,2,8) a(7,3,8) a(4,1,9) a(9,2,9) a(1,3,9) a(6,4,1) a(1,5,1) a(9,6,1) a(8,4,2) a(6,5,2) a(3,6,2) a(4,4,3) a(7,5,3) a(2,6,3) a(9,4,4) a(4,5,4) a(1,6,4) a(1,4,5) a(5,5,5) a(8,6,5) a(7,4,6) a(3,5,6) a(5,6,6) a(3,4,7) a(8,5,7) a(6,6,7) a(2,4,8) a(9,5,8) a(4,6,8) a(5,4,9) a(2,5,9) a(7,6,9) a(2,7,1) a(4,8,1) a(7,9,1) a(9,7,2) a(2,8,2) a(4,9,2) a(8,7,3) a(3,8,3) a(5,9,3) a(7,7,4) a(5,8,4) a(3,9,4) a(4,7,5) a(7,8,5) a(2,9,5) a(1,7,6) a(9,8,6) a(6,9,6) a(5,7,7) a(1,8,7) a(9,9,7) a(6,7,8) a(8,8,8) a(1,9,8) a(3,7,9) a(6,8,9) a(8,9,9)<br>SATISFIABLE<br>Models : 1<br>Calls : 1<br>Time : 0.240s (Solving: 0.21s 1st Model: 0.09s Unsat: 0.12s)<br>CPU Time : 0.219s |

Problem 8

**Input Program**

```
% Problem 8: Maximum Bishops on n×n chessboard
#const n=8.
#const k=14.

% Generate: place exactly k bishops on the board
% bishop(R,C) means there is a bishop at position (R,C)
{ bishop(R,C) : R=1..n, C=1..n } = k.

% Test: No two bishops on the same "/" diagonal (R+C constant)
% Two cells (R1,C1) and (R2,C2) are on the same "/" diagonal if R1+C1 = R2+C2
:- bishop(R1,C1), bishop(R2,C2), R1+C1 == R2+C2, (R1,C1) != (R2,C2).

% Test: No two bishops on the same "\" diagonal (R-C constant)
% Two cells (R1,C1) and (R2,C2) are on the same "\" diagonal if R1-C1 = R2-C2
:- bishop(R1,C1), bishop(R2,C2), R1-C1 == R2-C2, (R1,C1) != (R2,C2).

% Display only the bishop positions
#show bishop/2.
```

**Command Line**

You should write 6 command lines below, one for each value of n from the set {3, …, 8}.
Hint 1: you do not need to find all stable models.
Hint 2: the value of k (which denotes the number of bishops) in each command line should be the biggest value that you can assign to have at least one stable model.

```
clingo problem8.txt -c n=3 -c k=4 0
clingo problem8.txt -c n=4 -c k=6 0
clingo problem8.txt -c n=5 -c k=8 0
clingo problem8.txt -c n=6 -c k=10 0
clingo problem8.txt -c n=7 -c k=12 0
clingo problem8.txt -c n=8 -c k=14 0
```

**Output of clingo**

Since there are 6 outputs for 6 different values of n, please do not copy and paste the outputs of clingo for Problem 8 (and also for Problem 2 earlier).

**Answer to Questions**

Draw a table that lists the maximum value of bishops when the chessboard is n by n, where n is 3, 4, 5, 6, 7, 8. Infer the general function f(n) for n >= 3 that returns the maximum value of bishops.

| Value of n | f(n) |
|---|---|
| 3 | 4 |
| 4 | 6 |
| 5 | 8 |
| 6 | 10 |
| 7 | 12 |
| 8 | 14 |

f(n) = 2(n - 1)

Problem 9

<table>
<tr><td>Input<br>Program</td><td>% Problem 9: Almost-Schur numbers A(k)<br>#const n=1.<br>#const k=1.<br><br>num(1..n).<br>col(1..k).<br><br>% Partition {1,...,n} into k subsets:<br>% in(X,C) means number X is in set C<br>1 { in(X,C) : col(C) } 1 :- num(X).<br><br>% Almost sum-free constraint:<br>% If A and B are DIFFERENT (A < B) in the same set C,<br>% then A+B cannot also be in C (when A+B is within 1..n).<br>:- in(A,C), in(B,C), A < B, S = A + B, S <= n, in(S,C).<br><br>% Optional symmetry breaking (safe; speeds up search):<br>in(1,1).<br><br>#show in/2.</td></tr>
<tr><td>Command<br>Line</td><td>You should write 4 command lines below, one for each value of k from the set {1, 2, 3, 4}.<br>Hint 1: you do not need to find all stable models.<br>Hint 2: the value of n in each command line should be the biggest value that you can assign to have at least one stable model.<br>clingo problem9.txt -c k=1 -c n=2 0<br>clingo problem9.txt -c k=2 -c n=8 0<br>clingo problem9.txt -c k=3 -c n=23 0<br>clingo problem9.txt -c k=4 -c n=66 0</td></tr>
<tr><td>Output<br>of clingo</td><td>There are 4 outputs for 4 different values of k. Please copy and paste the outputs for k in {1, 2, 3} in the following table.<br><br>
<table>
<tr><th>Value of k</th><th>Output</th></tr>
<tr><td>1</td><td>clingo version 5.8.0<br>Reading from problem9.txt<br>Solving...<br>Answer: 1 (Time: 0.002s)<br>in(1,1) in(2,1)<br>SATISFIABLE<br><br>Models : 1<br>Calls : 1<br>Time : 0.007s (Solving: 0.01s 1st Model: 0.00s Unsat: 0.01s)<br>CPU Time : 0.016s</td></tr>
<tr><td>2</td><td>clingo version 5.8.0<br>Reading from problem9.txt<br>Solving...<br>Answer: 1 (Time: 0.001s)<br>in(1,1) in(2,1) in(3,2) in(4,1) in(5,2) in(6,2) in(7,2) in(8,1)</td></tr>
</table>
</td></tr>
</table>

| | | |
|---|---|---|
| | | SATISFIABLE<br><br>Models : 1+<br>Calls : 1<br>Time : 0.006s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)<br>CPU Time : 0.000s |
| | 3 | clingo version 5.8.0<br>Reading from problem9.txt<br>Solving...<br>Answer: 1 (Time: 0.004s)<br>in(1,1) in(2,1) in(3,2) in(4,1) in(5,2) in(6,2) in(7,2) in(8,1) in(9,3) in(10,3) in(11,1) in(12,3) in(13,3) in(14,3) in(15,3) in(16,3) in(17,1) in(18,3) in(19,2) in(20,3) in(21,2) in(22,1) in(23,2)<br>SATISFIABLE<br><br>Models : 1+<br>Calls : 1<br>Time : 0.010s (Solving: 0.01s 1st Model: 0.00s Unsat: 0.00s)<br>CPU Time : 0.000s |

Answer to Questions

Fill in the values accordingly.

| | |
|---|---|
| Exact value of A(1) | 2 |
| Exact value of A(2) | 8 |
| Exact value of A(3) | 23 |
| Largest lower bound for A(4)<br>Note: it would take longer time when you increase the value of n. Thus, you may stop increasing the value of n when your program does not terminate within 10 minutes and submit the last trial of n. | 66 (SATISFIABLE for n=66; n=67 did not finish within time limit / was interrupted). |